

complete-summary

Project Manual: Lunar Elemental Mapping with Chandrayaan-2

This document provides a complete overview of the project, from the scientific objectives and underlying physics to the detailed coding procedures and debugging history.

1. Project Description

This project presents a complete, end-to-end Python pipeline for processing raw X-ray data from India's Chandrayaan-2 mission. It analyzes synchronized data from the CLASS (lunar spectrometer) and XSM (solar monitor) instruments to generate scientific maps of elemental abundances on the lunar surface. The primary scientific goal is to derive and map the Mg/Si and Al/Si abundance ratios, which are key indicators of the Moon's geological composition and history.

Key Features

- **Automated Pipeline:** A series of scripts automates the entire workflow, from raw data ingestion to final map generation.
 - **Professional Calibration:** Uses the `XSPEC` astrophysics software (via the `pyxspec` Python library) for robust, forward-fitting-based calibration of the solar spectrum.
 - **Scientific Signal Isolation:** Implements a model-and-subtract technique to scientifically remove the dominant scattered solar continuum and isolate the faint X-ray Fluorescence (XRF) signal from the Moon.
 - **Accurate Area-Weighted Mapping:** Generates final maps using a scientifically accurate, equal-area **HEALPix** grid to properly handle overlapping observation footprints and avoid polar distortion.
 - **Interactive 3D Visualization:** Includes scripts to create interactive, rotatable 3D maps of the final data points using `plotly`.
-

2. Definitive Analysis Procedure

The workflow is divided into three major phases: data preparation, batch analysis, and final map generation.

Phase 1: Data Preparation & Filtering

This phase converts thousands of raw instrument files into a single, clean list of high-quality observations.

- **Step 1.1: Extract CLASS Metadata**

- **Script:** `perMonthEachDay_extract_class_metadata.py`
- **Objective:** To catalog the time, location, and geometry of every lunar observation.
- **Inputs:** Raw CLASS `.fits` files.
- **Output:** Daily CSV files of CLASS metadata.

- **Step 1.2: Extract XSM Metadata**

- **Script:** `new_extract_xsm_metadata.py`
- **Objective:** To catalog the per-second brightness of the Sun in X-rays.
- **Inputs:** Raw daily XSM `.fits` file.
- **Output:** A detailed per-second CSV file for the day.

- **Step 1.3: Temporally Match CLASS & XSM Data**

- **Script:** `match_CLASS_XSM.py`
- **Objective:** To link each brief CLASS observation to the solar conditions at that exact moment.
- **Inputs:** Daily CLASS and XSM metadata CSVs.
- **Output:** A new, combined CSV file for the day.

- **Step 1.4: Filter for High-Quality Observations**

- **Script:** `diagnose_and_filter_match_v1.2.py`
- **Objective:** To remove scientifically unusable data.

- **Inputs:** The matched CSV file.
- **Output:** A `..._filtered.csv` file containing only good data.
- **Step 1.5: Create Premium Observation Subset**
 - **Script:** `filteredFiles_subset_basedOnSolarAng.py`
 - **Objective:** To isolate the absolute best-quality data for analysis.
 - **Inputs:** The `..._filtered.csv` file.
 - **Output:** The final input for the main analysis, `match_CLASS_XSM_20250801_prime.csv`.

Phase 2: Automated Scientific Analysis (The Batch Run)

This is the core of the analysis, controlled by the main batch script.

- **Main Script:** `run_analysis_batch.py`
- **Objective:** To derive the elemental abundance ratios for every observation in the `...prime.csv` file.
- **Procedure:** The script loops through every observation and executes the following pipeline:
 1. **Prepare XSM Spectrum (`prepare_xsm_for_xspec.py`):** Creates a time-averaged solar spectrum in the required `.pha` format.
 2. **Calibrate Solar Spectrum (Forward-Fitting):** Uses `pyxspec` to fit a physical model to the XSM data, removing instrumental effects to find the true solar flux `F_sun(E)`.
 3. **Isolate Lunar Signal (Model-and-Subtract):** Uses `F_sun(E)` to model and subtract the scattered solar continuum from the CLASS spectrum.
 4. **Measure Elemental Fingerprints (`py_fit_xrf_lines.py`):** Fits a multi-Gaussian model to the peaks in the clean residual spectrum to measure the line flux of Mg, Al, and Si.
- **Output:** A master CSV file, `final_abundance_ratios.csv`.

Phase 3: Final Map Generation

This final phase visualizes the scientific results.

- **Step 3.1: Consolidate Data**

- **Script:** `merge_results.py`
- **Objective:** To merge the scientific results with the full footprint area data.
- **Inputs:** `final_abundance_ratios.csv` and `...prime.csv` .
- **Output:** `final_abundance_ratios_complete.csv` .

- **Step 3.2: Create the Final Map**

- **Script:** `plot_healpix_map.py`
 - **Objective:** To create a scientifically accurate, area-preserving map.
 - **Inputs:** The `..._complete.csv` file.
 - **Output:** The final map images (e.g., `lunar_healpix_map_advanced_mg_si_ratio.png`).
-

3. The Underlying Physics

The measurement of elemental abundances is a chain of physical processes that begins at the Sun.

- **Source: Solar Coronal X-rays:** The Sun's corona, a plasma at millions of Kelvin, produces X-rays through two mechanisms: **Bremsstrahlung** (a continuous spectrum from decelerating electrons) and **Line Emission** (discrete energy photons from de-exciting ionized atoms). This combined spectrum, `F_sun(E)` , travels to the Moon.
- **Interaction with Lunar Regolith:** When a solar photon hits the Moon's surface dust (the regolith), it can either scatter off an atom (the "noise") or be absorbed via **Photoelectric Absorption** (the "signal"). This absorption ejects an inner-shell electron, leaving the atom in an unstable state.
- **Response: X-ray Fluorescence (XRF):** The unstable atom immediately relaxes. An outer-shell electron "falls" into the inner-shell vacancy, emitting a new X-ray photon. The energy of this photon is discrete and unique to the element, creating a **characteristic "fingerprint"**. For example, the K-alpha line of Silicon is always ~1.74 keV.

- **Measurement:** The CLASS detector measures these fluorescent X-rays. Its physical properties modify the signal: the **Effective Area (ARF)** describes its sensitivity, and the **Energy Dispersion (RMF)** describes its "blurriness." Accounting for these effects is crucial for accurate analysis.
-

4. Instruments, Data, and Calibration Files

CLASS (Chandrayaan-2 Large Area Soft X-ray Spectrometer)

- **Measures:** X-rays fluorescing from the **Moon's surface**.
- **Data:** Its primary data product is a **spectrum** (a histogram of X-ray counts vs. energy) stored in a `.fits` file. The header contains critical metadata like time, exposure, and the footprint's location (`BORE_LAT`, `BORE_LON`, `v0_lat`, etc.).
- **Calibration Files:**
 - `class_arf_v1.arf` (**ARF**): The "sensitivity" file. Used in `py_forward_model_and_subtract.py`.
 - `background_allevvents.fits` (**BKG**): The "instrumental noise" file. Used in `py_forward_model_and_subtract.py`.

XSM (Solar X-ray Monitor)

- **Measures:** The full spectrum of X-rays coming from the **Sun**, providing a new spectrum every second.
 - **Data:** A time-series of spectra stored in a single large, daily `.fits` file. The key data is in the `DataArray` column.
 - **Calibration Files:**
 - `...rsp` (**RSP**): The "Response" file, containing both the "blur" (RMF) and "sensitivity" (ARF) information.
 - `...pha` (**BKG**): The instrumental background file for XSM.
 - Both files are used by the `pyxspec` analysis step inside `run_analysis_batch.py`.
-

5. Tools and Libraries

External Software

- **HEASOFT & XSPEC:** A professional astrophysics software package. `XSPEC` is the global standard for X-ray spectral fitting and was used for the critical **forward-fitting** step to calibrate the solar spectrum.

Python Libraries

- `pathlib` : For modern, object-oriented handling of file and directory paths.
 - `pandas` : The backbone for all data manipulation, handling the reading, filtering, and merging of CSV files.
 - `numpy` : For all numerical calculations, including array mathematics and spectral rebinning.
 - `astropy` : For handling astronomy-specific file formats (`FITS`) and data types (`Time` objects).
 - `scipy` : For scientific fitting (`curve_fit` , `minimize`) and interpolation (`interp1d`).
 - `matplotlib` : For creating 2D diagnostic and final plots.
 - `healpy` : For creating and plotting the final, scientifically accurate **HEALPix** maps.
 - `plotly` : For creating interactive 3D visualizations of the data.
-

6. Online Resources and Websites

- **ISRO PRADAN Archive (pradan.issdc.gov.in):** The primary source for all scientific data (CLASS and XSM `.fits` files) and the official calibration files.
 - **LROC Science Operations Center (lroc.sese.asu.edu):** Source for the high-resolution lunar base map used for visualization.
 - **HEASARC (heasarc.gsfc.nasa.gov):** Source for the essential `HEASOFT` / `XSPEC` scientific analysis software.
 - **Python Package Index (PyPI) (pypi.org):** The source for all open-source Python libraries used in the project.
-

7. Project Directory Structure

The project uses a structured layout to separate raw data, calibration files, intermediate metadata, scripts, and final results.

```
/moon_project/
├── calib/      # Stores instrument calibration files (RMF, ARF, BKG).
├── data/       # Stores raw CLASS and XSM .fits data.
├── maps/       # Stores the lunar base map image for plotting.
├── metadata/   # Stores intermediate CSV catalogs from Phase 1.
├── plots/      # Stores all final visual outputs (PNG maps, HTML 3D plots).
├── results/    # Stores all final data outputs from the analysis.
├──   └── batch_run/ # Contains intermediate files and the final CSV.
└── scripts/    # Contains all Python source code for the pipeline.
```

8. Project Debugging and Problem Solving

This section chronicles the major challenges and their solutions.

- **Problem: Python Environment Conflicts.**
 - **Symptom:** Crashes due to incompatible library versions (`numpy.dtype size changed`).
 - **Solution:** A dedicated **Python virtual environment** (`venv`) was created to isolate project dependencies.
- **Problem: Silent `XSPEC` Failures.**
 - **Symptom:** `XSPEC` would start and quit with no errors, producing empty output.
 - **Diagnosis:** Incorrect command-line invocation and bad file line endings.
 - **Solution:** Corrected the command to use input redirection (`xspec < script.cmd`) and used `dos2unix` to fix file formats.
- **Problem: Incomplete `.pha` Headers.**
 - **Symptom:** `XSPEC` failed to load generated `.pha` files, complaining of missing keywords.
 - **Diagnosis:** The script was not generating fully OGIP-compliant FITS headers.

- **Solution:** The `prepare_xsm_for_xspec.py` script was updated to write all required keywords and full paths to calibration files in the header.
 - **Problem:** `pyxspec` Failures.
 - **Symptom:** `AttributeError` crashes or silent failures producing empty data arrays.
 - **Diagnosis:** A combination of incorrect method calls and, most importantly, scientifically failed fits (e.g., zero normalization) that `pyxspec` did not report as errors.
 - **Solution:** The `pyxspec` function was rewritten to use the correct syntax and, crucially, a **diagnostic check** was added after the fit to verify that it produced a physically meaningful result.
-

9. Glossary of Key Terms

- **Solar Angle (`solarang_deg`):** The angle of the Sun in the sky at the observation point on the Moon. Low angles mean better illumination.
- **Forward-Fitting:** An analysis method where you create a perfect theoretical model, "blur" it with known instrumental effects, and adjust the model until the blurred version matches the real, observed data.
- **Spectrum:** A graph showing the intensity of radiation at different energies. The peaks reveal the presence of specific elements.
- **Continuum:** The smooth, underlying baseline of a spectrum, caused by scattered X-rays.
- **Line Flux:** The total strength of an emission line, measured by the area under its peak. This is proportional to elemental abundance.
- **Footprint:** The specific area on the Moon's surface observed by the instrument at one time.
- **Gridding / Binning:** Creating a map by dividing a surface into a grid and averaging all data points that fall into each cell.
- **HEALPix:** A professional method for dividing a sphere into pixels that all have the exact same surface area, preventing distortion in maps.